

Sure 👍 — here's a **comprehensive list of MySQL interview questions with detailed answers**, grouped by **difficulty level** so you can prepare effectively.

🟢 Beginner-Level MySQL Questions

1. What is MySQL?

Answer:

MySQL is an open-source relational database management system (RDBMS) that stores data in tables made up of rows and columns. It uses **SQL (Structured Query Language)** to manage and manipulate data.

It is widely used in web applications due to its **speed, reliability, and ease of use**.

2. What are the main differences between SQL and MySQL?

Feature	SQL	MySQL
Definition	Language used to query and manage databases	RDBMS that implements SQL
Function	Used to access, manipulate, and manage data	Stores and manages databases
Example	<code>SELECT * FROM employees;</code>	Runs and processes the query above

3. What are the different data types in MySQL?

Answer:

- **Numeric types:** INT, BIGINT, FLOAT, DOUBLE, DECIMAL
- **String types:** CHAR, VARCHAR, TEXT, BLOB
- **Date and time types:** DATE, TIME, DATETIME, TIMESTAMP

- **Boolean type:** BOOL (alias for TINYINT(1))

4. What is the difference between CHAR and VARCHAR ?

Feature	CHAR	VARCHAR
Storage	Fixed length	Variable length
Performance	Faster for fixed-size data	More efficient for variable-size data
Example	CHAR(10) always stores 10 chars	VARCHAR(10) stores up to 10 chars

5. What is a Primary Key?

Answer:

A **Primary Key** uniquely identifies each record in a table.

- It cannot be **NULL**.
- It must be **unique** for every record.
- Each table can have only **one primary key**.

Example:

sql

 Copy code

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

6. What is a Foreign Key?

Answer:

A **Foreign Key** is a field that links one table to another. It enforces **referential integrity** by ensuring that a value in one table matches a value in another.

Example:

sql

 Copy code

```
CREATE TABLE department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
  
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)  
);
```

● Intermediate-Level MySQL Questions

7. What are Joins in MySQL?

Answer:

Joins combine rows from two or more tables based on a related column.

Types:

- **INNER JOIN** – Returns matching rows from both tables.
- **LEFT JOIN** – Returns all rows from the left table, even if no match exists.
- **RIGHT JOIN** – Returns all rows from the right table.
- **FULL JOIN** – Returns all rows from both tables (not supported directly in MySQL; use UNION).
- **CROSS JOIN** – Returns the Cartesian product of both tables.

Example:

sql

 Copy code

```
SELECT e.name, d.dept_name  
FROM employee e  
INNER JOIN department d ON e.dept_id = d.dept_id;
```

8. What are indexes in MySQL?

Answer:

An **index** improves query performance by allowing faster data retrieval.

However, it can **slow down write operations** (INSERT, UPDATE, DELETE).

Example:

sql

 Copy code

```
CREATE INDEX idx_name ON employee(name);
```

9. What is the difference between WHERE and HAVING ?

Clause	Used With	Filters
WHERE	SELECT, UPDATE, DELETE	Rows before grouping
HAVING	SELECT with GROUP BY	Groups after aggregation

Example:

sql

 Copy code

```
SELECT dept_id, COUNT(*)  
FROM employee  
GROUP BY dept_id  
HAVING COUNT(*) > 5;
```

10. What is normalization? What are its types?

Answer:

Normalization organizes data to eliminate redundancy and improve integrity.

Common normal forms:

1. **1NF**: Each cell holds one value.
2. **2NF**: 1NF + no partial dependency.

3. 3NF: 2NF + no transitive dependency.
4. BCNF: Stronger form of 3NF.

11. What is a transaction in MySQL?

Answer:

A **transaction** is a group of SQL statements executed as a single unit.

It follows **ACID** properties:

- **Atomicity** – All or none
- **Consistency** – Preserves data integrity
- **Isolation** – Independent transactions
- **Durability** – Results persist after commit

Example:

```
sql
```

 Copy code

```
START TRANSACTION;  
UPDATE account SET balance = balance - 500 WHERE id = 1;  
UPDATE account SET balance = balance + 500 WHERE id = 2;  
COMMIT;
```

Advanced-Level MySQL Questions

12. What is the difference between DELETE , TRUNCATE , and DROP ?

Command	Description	Rollback	Removes Structure
DELETE	Deletes specific rows	 Yes	 No
TRUNCATE	Deletes all rows (faster)	 No	 No
DROP	Deletes entire table	 No	 Yes

13. Explain the concept of MySQL storage engines.

Answer:

MySQL supports multiple **storage engines**, each with different capabilities.

Engine	Features
InnoDB	Supports transactions, row-level locking, foreign keys
MyISAM	Fast for read-heavy workloads, no transactions
MEMORY	Stores data in RAM, very fast but volatile
CSV	Stores data in CSV format

Check engine:

```
sql
```

[Copy code](#)

```
SHOW TABLE STATUS WHERE Name = 'employee';
```

14. What are triggers in MySQL?

Answer:

A **trigger** automatically executes before or after an INSERT, UPDATE, or DELETE.

Example:

```
sql
```

[Copy code](#)

```
CREATE TRIGGER before_insert_employee
BEFORE INSERT ON employee
FOR EACH ROW
SET NEW.created_at = NOW();
```

15. What are stored procedures and functions?

Answer:

- **Stored Procedure:** A set of SQL statements stored in the database for reuse.
- **Function:** Similar, but always returns a value.

Example:

sql

 Copy code

```
CREATE PROCEDURE getEmployees()  
BEGIN  
    SELECT * FROM employee;  
END;
```

16. How do you optimize a slow MySQL query?

Answer:

- Use **EXPLAIN** to analyze query execution.
 - Create **indexes** on frequently searched columns.
 - Avoid **SELECT ***.
 - Optimize **JOINS** and **subqueries**.
 - Use **LIMIT** for pagination.
 - Partition large tables if needed.
-

17. How to perform database backup and restore in MySQL?

Answer:

- **Backup:**

bash

 Copy code

```
mysqldump -u root -p database_name > backup.sql
```

- **Restore:**

bash

 Copy code

```
mysql -u root -p database_name < backup.sql
```

Would you like me to prepare a **PDF cheat sheet** of these questions with concise bullet answers (perfect for quick revision before an interview)?



questions on keys

Perfect 🍌 — let's go deep into **MySQL interview questions on Keys** with **detailed answers** and **examples**.

Keys are one of the most common and important MySQL interview topics.

🔑 MySQL Keys – Interview Questions and Answers

1. What is a Key in MySQL?

Answer:

A **key** is a constraint used to **uniquely identify** a record or establish a **relationship** between tables.

Keys ensure **data integrity** and **query performance**.

Types of Keys in MySQL:

1. Primary Key
 2. Foreign Key
 3. Unique Key
 4. Candidate Key
 5. Composite Key
 6. Alternate Key
 7. Super Key
-

2. What is a Primary Key?

Answer:

A **Primary Key** uniquely identifies each record in a table.

- It cannot be **NULL**.
- It must be **unique**.
- Each table can have **only one primary key**, but it can consist of **multiple columns** (composite key).

Example:

sql

 Copy code

```
CREATE TABLE students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(50),  
  age INT  
);
```

Or composite primary key:

sql

 Copy code

```
CREATE TABLE exam_results (  
  student_id INT,  
  subject_id INT,  
  marks INT,  
  PRIMARY KEY (student_id, subject_id)  
);
```

3. What is a Foreign Key?

Answer:

A **Foreign Key** creates a relationship between two tables.

It ensures that a value in one table matches a value in another table (referential integrity).

Example:

sql

 Copy code

```
CREATE TABLE departments (  
  dept_id INT PRIMARY KEY,  
  dept_name VARCHAR(50)  
);  
  
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY,  
  emp_name VARCHAR(50),  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

If you try to insert an employee with a dept_id that doesn't exist in departments , MySQL will reject it.

4. What is a Unique Key?

Answer:

A Unique Key ensures that all values in a column are **unique**, but unlike the primary key, it can contain **NULL values** (only one NULL per column).

Example:

sql

Copy code

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY,  
  email VARCHAR(100) UNIQUE  
);
```

Difference between Primary Key and Unique Key:

Feature	Primary Key	Unique Key
Uniqueness	Ensures unique values	Ensures unique values
NULL allowed	✗ No	✓ Yes (one NULL allowed)

Feature	Primary Key	Unique Key
Count per table	Only one	Can have multiple

5. What is a Candidate Key?

Answer:

A **Candidate Key** is any column or combination of columns that can **uniquely identify** a row in a table.

Among candidate keys, **one is chosen as the Primary Key**.

Example:

sql

 Copy code

```
CREATE TABLE employees (  
    emp_id INT,  
    email VARCHAR(100),  
    phone VARCHAR(15)  
);
```

Here:

- emp_id , email , and phone can all uniquely identify an employee.

So, they are **candidate keys**.

If you choose emp_id as the **primary key**, others become **alternate keys**.

6. What is an Alternate Key?

Answer:

An **Alternate Key** is a candidate key that is not chosen as the primary key.

Example:

If both email and emp_id are candidate keys, and emp_id is selected as the **primary key**, then email becomes an **alternate key**.

7. What is a Composite Key?

Answer:

A **Composite Key** is a key made up of **two or more columns** used together to uniquely identify a record.

Example:

sql

 Copy code

```
CREATE TABLE order_details (  
  order_id INT,  
  product_id INT,  
  quantity INT,  
  PRIMARY KEY (order_id, product_id)  
);
```

Here, neither `order_id` nor `product_id` alone is unique, but together they uniquely identify each record.

8. What is a Super Key?

Answer:

A **Super Key** is any combination of columns that can uniquely identify a record. It may contain **extra attributes** beyond what's necessary.

Example:

If `(emp_id)` is unique, then `(emp_id, emp_name)` is also a **super key**, but not a **candidate key** (because it has extra column).

9. Can a table have multiple Primary Keys?

Answer:

 No.

A table can have **only one primary key**, but it may consist of **multiple columns** (composite key).

10. Can a Foreign Key reference a non-primary key column?

Answer:

✓ Yes, as long as that column has a **unique constraint**.

Example:

sql

 Copy code

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY,  
  email VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  user_email VARCHAR(100),  
  FOREIGN KEY (user_email) REFERENCES users(email)  
);
```

11. Can a table have multiple Foreign Keys?

Answer:

✓ Yes.

A table can have **multiple foreign keys**, each referring to a different table.

12. What happens if you delete a row in the parent table having a foreign key reference?

Answer:

It depends on the **ON DELETE** rule:

- ON DELETE CASCADE – deletes the child rows automatically.
- ON DELETE SET NULL – sets child's foreign key column to NULL.
- ON DELETE RESTRICT – prevents deletion if child rows exist (default).

Example:

sql

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE  
);
```

13. What is the difference between Primary Key and Foreign Key?

Feature	Primary Key	Foreign Key
Purpose	Uniquely identifies each record	Establishes a relationship between tables
Uniqueness	Must be unique	Can have duplicate values
NULL allowed	Not allowed	Allowed
Table count	One per table	Can have many per table

14. Can we define both a Primary Key and a Unique Key on the same column?

Answer:

Technically yes, but it's **redundant** — both enforce uniqueness. Usually, we use **either one**.

15. What's the difference between Composite Key and Compound Key?

Answer:

They are often used interchangeably.

However, in strict terms:

- **Composite Key:** Combination of two or more columns used as **primary key**.
- **Compound Key:** Combination of columns that uniquely identify records but may **not** be a **primary key**.

16. What is the purpose of indexing keys?

Answer:

- Speeds up query performance.
- Helps MySQL locate rows quickly using keys.
- Especially useful for **foreign keys** and **columns used in WHERE/JOIN**.

Example:

sql

 Copy code

```
CREATE INDEX idx_dept_id ON employees(dept_id);
```